

hw1ktx887

August 26, 2026

1 HW1 AW KTX887 — Assignment turned in as pdf; not a replacement of actual work

1.1 Dirac delta in orthogonal coordinates

short computational check of Problems 1.2–1.3. The numerics do not replace the distributional proof i provided – only test the two claims.

1. the Gaussian sequence samples a smooth test function at the source point;
2. each proposed charge density has the required total charge.

```
[12]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from numpy.polynomial.hermite import hermgauss
import seaborn as sns
sns.set_style("darkgrid")

#params
q0 = np.array([0.30, -0.40, 0.20]) #(u', v', w')
metric_factors = np.array([1.40, 0.80, 1.70]) #(U', V', W')
alphas = 2.0 ** -np.arange(1, 8)

Q = 7.0
lambda_line = 2.5
R = 2.3
b = 1.4
```

1.2 1.2 — Gaussian sequence

Locally,

$$ds^2 = \frac{du^2}{U'^2} + \frac{dv^2}{V'^2} + \frac{dw^2}{W'^2}, \quad d^3x = \frac{du \, dv \, dw}{UVW}.$$

Thus $D_\alpha d^3x$ is locally the product of three normalized Gaussians, with coordinate widths $\alpha U'$, $\alpha V'$, and $\alpha W'$. For any smooth f ,

$$\int D_\alpha f d^3x \longrightarrow f(u', v', w').$$

Gauss–Hermite (from numpy) quadrature evaluates that integral directly.

```
[13]: def test_function(u, v, w):
    envelope = np.exp(-0.20 * (u**2 + 2.0*v**2 + 0.50*w**2))
    return envelope * (1.0 + 0.10*u - 0.05*v + 0.03*u*w)

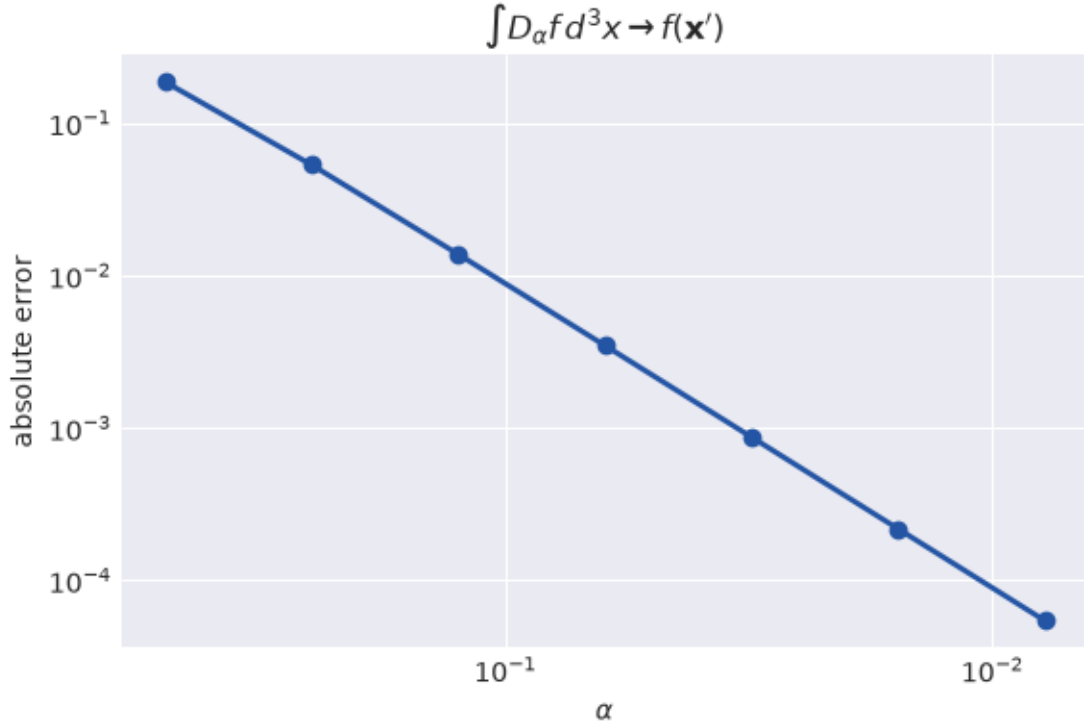
def gaussian_action(alpha, order=24):
    #bc E[f(q0 + alpha * metric_factors * Z)], Z ~ N(0, I)
    nodes, weights = hermgauss(order)
    u = q0[0] + np.sqrt(2.0) * alpha * metric_factors[0] * nodes[:, None, None]
    v = q0[1] + np.sqrt(2.0) * alpha * metric_factors[1] * nodes[None, :, None]
    w = q0[2] + np.sqrt(2.0) * alpha * metric_factors[2] * nodes[None, None, :]
    W3 = weights[:, None, None] * weights[None, :, None] * weights[None, None, :]
    ↪
    return np.sum(W3 * test_function(u, v, w)) / np.pi**1.5

target = test_function(*q0)
actions = np.array([gaussian_action(alpha) for alpha in alphas])
errors = np.abs(actions - target)

pd.DataFrame({
    'alpha': alphas,
    'Gaussian action': actions,
    'f(source)': target,
    'absolute error': errors,
})
```

	alpha	Gaussian action	f(source)	absolute error
0	0.500000	0.779348	0.965126	0.185778
1	0.250000	0.912101	0.965126	0.053025
2	0.125000	0.951387	0.965126	0.013739
3	0.062500	0.961659	0.965126	0.003466
4	0.031250	0.964257	0.965126	0.000869
5	0.015625	0.964908	0.965126	0.000217
6	0.007812	0.965071	0.965126	0.000054

```
[14]: fig, ax = plt.subplots(figsize=(6.5, 4.0))
ax.loglog(alphas, errors, 'o-', lw=2, color='#2455A4')
ax.invert_xaxis()
ax.set(xlabel=r'$\alpha$', ylabel='absolute error',
       title=r'$\int D_\alpha f, d^3x \to f(\mathbf{x}')$')
plt.show()
```



The error is $O(\alpha^2)$: this is expected as a normalized gaussian is an even function; so its linear moment should vanish.

1.3 1.3 — charge normalization

Let $g_\epsilon(t)$ be a normalized Gaussian approximation to $\delta(t)$ my derived densities are

$$\rho_a = \frac{Q}{4\pi R^2} \delta(r - R), \quad \rho_b = \frac{\lambda}{2\pi b} \delta(s - b),$$

$$\rho_c = \frac{Q}{\pi R^2} \delta(z) \Theta(R - s), \quad \rho_d = \frac{Q}{\pi R^2} \frac{\delta(\theta - \pi/2)}{r} \Theta(R - r).$$

computationally, the next cell replaces each delta by g_ϵ then computes the recovered charge and specified charge. std normal variable keeps each integral well resolved as $\epsilon \rightarrow 0$.

```
[15]: def phi(t):
    return np.exp(-0.5*t**2) / np.sqrt(2.0*np.pi)

t = np.linspace(-8.0, 8.0, 40_001)
epsilons = 2.0 ** -np.arange(0, 7)
rows = []

for eps in epsilons:
```

```

#physical radial coordinates require r,s >= 0.
shell_mask = (R + eps*t) >= 0.0
cyl_mask = (b + eps*t) >= 0.0
theta_mask = np.abs(eps*t) <= np.pi/2

shell_ratio = np.trapezoid(
    ((R + eps*t[shell_mask]) / R)**2 * phi(t[shell_mask]),
    t[shell_mask],
)
cylinder_ratio = np.trapezoid(
    ((b + eps*t[cyl_mask]) / b) * phi(t[cyl_mask]),
    t[cyl_mask],
)
flat_disc_ratio = np.trapezoid(phi(t), t)
spherical_disc_ratio = np.trapezoid(
    np.cos(eps*t[theta_mask]) * phi(t[theta_mask]),
    t[theta_mask],
)

rows.append([eps, shell_ratio, cylinder_ratio,
             flat_disc_ratio, spherical_disc_ratio])

charge_check = pd.DataFrame(
    rows,
    columns=['epsilon', 'sphere shell', 'cylinder surface',
            'disc: cylindrical', 'disc: spherical'],
)
charge_check

```

```

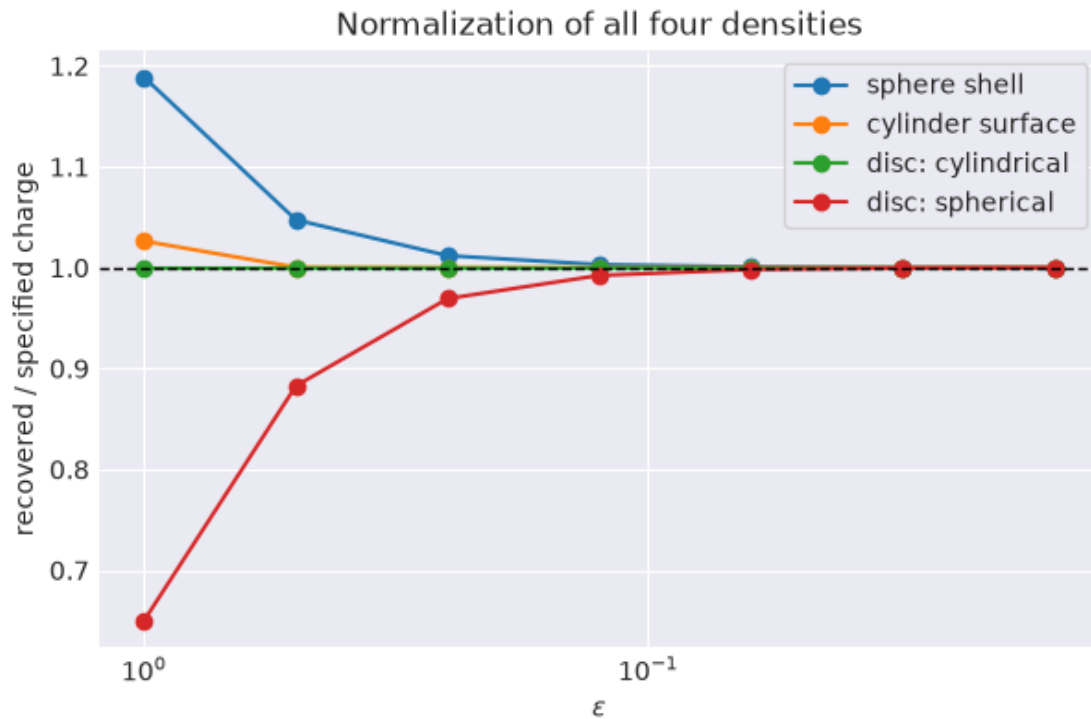
[15]:
   epsilon  sphere shell  cylinder surface  disc: cylindrical \
0  1.000000      1.188601      1.026192      1.0
1  0.500000      1.047259      1.000272      1.0
2  0.250000      1.011815      1.000000      1.0
3  0.125000      1.002954      1.000000      1.0
4  0.062500      1.000738      1.000000      1.0
5  0.031250      1.000185      1.000000      1.0
6  0.015625      1.000046      1.000000      1.0

   disc: spherical
0      0.650280
1      0.882723
2      0.969233
3      0.992218
4      0.998049
5      0.999512
6      0.999878

```

```
[17]: fig, ax = plt.subplots(figsize=(7.0, 4.2))
for column in charge_check.columns[1:]:
    ax.semilogx(charge_check['epsilon'], charge_check[column], 'o-', label=column)

ax.axhline(1.0, color='black', lw=1, ls='--')
ax.invert_xaxis()
ax.set(xlabel=r'$\epsilon$', ylabel='recovered / specified charge',
       title='Normalization of all four densities')
ax.legend(frameon=True)
plt.show()
```



i've shown that all four ratios tend to one. so small finite width bias in shells comes from averaging curved measures r^2 across the Gaussian $\rightarrow$ vanishes in the delta limit

```
[18]: # Short machine-checkable conclusion
assert errors[-1] < errors[0]
assert np.max(np.abs(charge_check.iloc[-1, 1:].to_numpy() - 1.0)) < 2.0e-4
print('Passed: sampling converges and every charge ratio tends to 1.')
```

Passed: sampling converges and every charge ratio tends to 1.

```
[ ]:
```